

A Machine Learning Approach to Expanding the Degrees of Freedom on Phone-Based Head Mounted Displays

Andrew J. Jones
Davidson College *

Raghuram Ramanujan
Davidson College †

Tabitha C. Peck
Davidson College ‡

ABSTRACT

Phone-based head mounted displays (HMDs) make participation in virtual reality (VR) easily accessible. However, phone-based HMDs can only track the user's head orientation, excluding head position. This not only stifles the user's immersion in the virtual environment, but also risks simulator sickness. To mitigate this limitation, we propose a neural network that predicts the user's head position given the head orientation history and other information about the user's movement that can be tracked from phone sensors. To analyze performance, the model iterates over the test dataset, predicting the entirety of the dataset's positional displacement history.

Index Terms: Human-centered computing, accessibility, machine learning, user centered design.

1 INTRODUCTION

Virtual reality (VR) is most accessible today through inexpensive hardware, such as smartphones attached to headsets as head-mounted displays (HMDs). This contrasts with more expensive VR hardware such as the Oculus Quest or the HTC Vive. Phone-based HMDs, despite their availability, have limited capabilities, as they can only track the user's head orientation, but not head position. In terms of degrees of freedom (DoF), phone-based HMDs track the user under a 3DoF tracking system—a 3-axis coordinate system describing the head's angular position, labeled *yaw* (the head looks left/right), *pitch* (the head looks up/down), and *roll* (the head tilts from side to side to the left/right, while the neck stays straight). Phones lack a 6DoF tracking system, which adds the body's movement forward/backward (*x*), up/down (*y*), and left/right (*z*) [3]. The use of a 3DoF tracking system instead of 6DoF can increase risk of simulator sickness. To enhance phone-based HMDs' tracking capabilities to a 6DoF system, we propose a supervised machine learning approach that reads the user's history of movement in a 3DoF system plus the user's *linear velocity*, *angular velocity*, and *linear acceleration*, which can be derived from a phone's gyroscope and accelerometer [4].

2 METHODS

2.1 Collecting Data for Machine Learning Model

We collected data from 11 seated participants (7 male, 4 female), each with a self-avatar. A view of the testing environment is shown in Figure 1. Head-tracking data (*time*, *x*, *y*, *z*, *yaw*, *pitch*, *roll*, *linear velocity*, *angular velocity*, *linear acceleration*) was collected at intervals of 10 ms for 5 minutes. Participants were encouraged to turn their heads and move

their torso while remaining in a seated position. The collected data was used for fitting and testing the model.

Initial Preprocessing: We removed discontinuities between Euler angle values (*yaw*, *pitch*, *roll*) using `numpy.unwrap()`. A Gaussian blur was applied to remove noise to all metrics except time. Every fifth sample was then retained. Linear acceleration was computed by taking the derivative of the linear velocity along each axis with respect to time (with `numpy.gradient()`). We replaced the raw values for each feature with the difference from the previous time step, except for *linear velocity*, *angular velocity*, and *linear acceleration* (with `numpy.ediff1d()`). We initialized the first row of each column to 0.

Train-Validation-Test Split and Data Treatment: We trimmed each participant's dataset to the size of the dataset with the least number of samples to expedite the preprocessing via NumPy's matrix operations. The last 1000 time steps of each participant's data were separated from the rest of the data to serve as the test set. The data was concatenated into a rank-3 tensor. To create examples where some rows are blank, we concatenated a padding of zeros to each slice of the tensor. The model's examples were made by iterating down the tensor along the second dimension, and extracting slices of 20 time steps at a time.



Figure 1: A view of the virtual environment where participant data was collected.

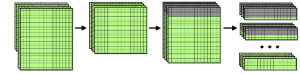


Figure 2: Diagram of the steps taken in preprocessing. The datasets are trimmed to the same size. After adding padding to the top of the tensor, we extracted slices from the tensor to form the model's examples.

After creating the examples, they were all concatenated into a new rank-3 tensor that was divided into training and validation datasets. Three-fourths of the slices were used to fit the model. The remaining slices were for validating the model's performance on unseen data after fitting the model.

The training data was then transformed using min-max scaling [2]. The statistics computed from the training data to perform this scaling were used to subsequently scale the validation and test data as well.

Input and Output: Each example has 20 time steps. However, the last ten changes in positional displacement are zeroed out before normalization, because those values are to be output by the model. The remaining segments of each example are the input data.

Model Architecture and Training: The model used was a multi-layer perceptron (MLP). The treatment of each example by the model is described in Figure 4. We used Adam, a first-order gradient-based optimization algorithm, to fit the model parameters via backpropagation. We trained the model for 100 epochs using a mean-squared-error (MSE) loss function.

*anjones1@davidson.edu

†raramanujan@davidson.edu

‡tapeck@davidson.edu

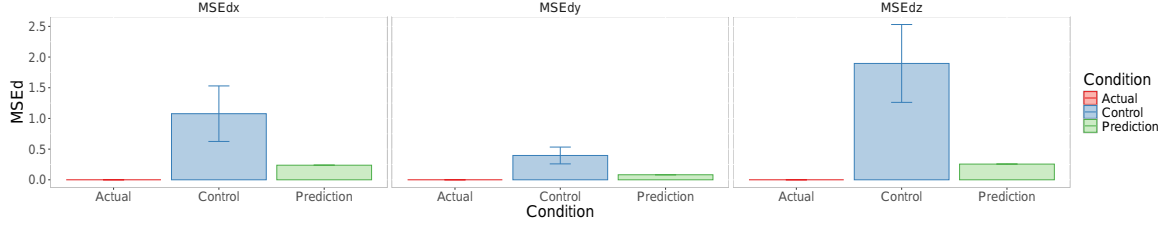


Figure 3: Differences in MSE compared against real positional displacement in all participants' test datasets; real displacement (actual), no displacement (control) and predicted displacement (prediction).

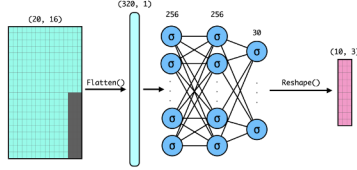


Figure 4: Architecture of the MLP, where each input to the model—being a 2D matrix—was flattened to a 1D vector, then passed through 2 dense hidden layers, each comprising 256 nodes applying the sigmoid activation function. The output layer contained 30 nodes (also applying a sigmoid activation) which were reshaped into a 10×3 matrix representing the model's prediction of the positional displacement values for the last 10 time steps.

Metrics: The model was used to generate predictions for the held out test data of each participant. Analysis compared the predicted positional data (x, y, z) to the actual positional data, as well as a control condition. The control condition applies no change in position from the original starting position, to mimic zero positional tracking. We computed the MSE between each sample and the actual positional data. The MSE for the actual data is zero implying that there is no deviation from the real positions.

3 RESULTS

Analysis was performed in R version 4.4.1. A 3 (Condition: Actual, Control, Prediction) $\times$ 3 (Axis: x, y, z) within-subjects ANOVA was used to evaluate the change in MSE for each change in position. Post-hoc analysis was performed using estimated marginal means.

A significant main effect of both Condition ($F(10, 1) = 7.48, p = .02, \eta^2 = .26$) and Axis ($F(17.2, 1.72) = 7.31, p = .007, \eta^2 = .07$) were found. The main effects were quantified by a higher order significant interaction ($F(17.27, 1.73) = 5.30, p = .02, \eta^2 = .10$). See Figure 3. The results of the posthoc tests are reported in Table 1.

4 DISCUSSION

The predicted track of the user's position is obtained by accumulating all the displacement values predicted by the model for each time step, beginning from the first time step in the user's data (excluding the padding). Despite accurate predictions of the differences, the predicted raw values of the user drifts over time in all axes.

5 CONCLUSIONS

We proposed an MLP neural network that predicts the user's position given the history of orientation, linear and angular velocities and linear acceleration. Although accurate in predicting differences, the raw values drift over time from the ground truth. Next steps involve changing the way data is presented during training.

contrast	estimate	SE	<i>t</i>	<i>p</i>
axis = MSEdx				
Prediction - Actual	0.2378	0.0037	63.999	<.0001
Prediction - Control	-0.8394	0.4541	-1.849	0.2038
Actual - Control	-1.0772	0.4524	-2.381	0.0897
axis = MSEdy				
Prediction - Actual	0.0822	0.0016	50.927	<.0001
Prediction - Control	-0.3142	0.1364	-2.303	0.1014
Actual - Control	-0.3964	0.1369	-2.896	0.0388
axis = MSEdz				
Prediction - Actual	0.2561	0.0031	82.569	<.0001
Prediction - Control	-1.6397	0.6338	-2.587	0.0643
Actual - Control	-1.8958	0.6344	-2.989	0.0333

P value adjustment: tukey method for comparing a family of 3 estimates
Table 1: Output from the contrasts for comparing Condition for each Axis.

The differences should be used to predict the output, but the raw values should be compared at the loss function so the model can then optimize its performance based on the raw values [1]. So far, we have not evaluated the model's impact on users. Thus, we also intend to use our model in a virtual environment and test its performance.

ACKNOWLEDGMENTS

This material is based upon work supported by the National Science Foundation under Grant 1942146. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation.

REFERENCES

- [1] T. Aykut, J. Xu, and E. Steinbach. Realtime 3d 360-degree telepresence with deep-learning-based head-motion prediction. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 9(1):231–244, 2019. doi: 10.1109/JETCAS.2019.2897220
- [2] K. Cabello-Solorzano, I. Ortigosa de Araujo, M. Peña, L. Correia, and A. J. Tallón-Ballesteros. The impact of data normalization on the accuracy of machine learning algorithms: A comparative analysis. In P. García Bringas, H. Pérez García, F. J. Martínez de Pisón, F. Martínez Álvarez, A. Troncoso Lora, Á. Herrero, J. L. Calvo Rolle, H. Quintián, and E. Corchado, eds., *18th International Conference on Soft Computing Models in Industrial and Environmental Applications (SOCO 2023)*, pp. 344–353. Springer Nature Switzerland, Cham, 2023.
- [3] Y.-K. Chang, M.-K. Chen, Y.-L. Li, H.-T. Li, and C.-K. Chiang. 6dof tracking in virtual reality by deep rnn model. In *2020 International Symposium on Computer, Consumer and Control (IS3C)*, pp. 193–196, 2020. doi: 10.1109/IS3C50286.2020.00057
- [4] I. A. Faisal, T. W. Purboyo, and A. S. R. Ansori. A review of accelerometer sensor and gyroscope sensor in imu sensors on motion capture. *J. Eng. Appl. Sci.*, 15(3):826–829, 2019.